

Document de Cadrage Agile et d'Architecture : Plateforme "Drink Safe"

Ce document définit le cadre méthodologique et technique du projet Drink Safe, visant à automatiser l'analyse de potabilité de l'eau via une architecture robuste et sécurisée.

Acteur

Action

Contrainte

1. Étape 1 : Analyse du Besoin et Contexte Métier

Le projet Drink Safe a pour mission principale de **prédire la potabilité de l'eau** à travers une **plateforme numérique centralisée**. Le système repose sur l'**analyse physico-chimique de prélèvements et l'ingestion automatisée de rapports de laboratoire via une technologie OCR** (Optical Character Recognition).

Cette solution, destinée aux collectivités territoriales, doit **répondre à des contraintes critiques de disponibilité, de confidentialité et de respect des normes sanitaires internationales**.

Tableau des Parties Prenantes

Profil, Rôle, Objectif

Agent de terrain, Utilisateur final, **Action** Déposer des mesures et obtenir un verdict de potabilité immédiat sans saisie manuelle.

Analyste Qualité, Superviseur métier, **Action** Surveiller la cohérence des prélèvements et traquer la dérive (drift) de l'IA via MLflow.

Responsable d'Exploitation, Garant technique, Assurer le **Action** maintien en condition opérationnelle (MCO) et la **gestion sécurisée des accès**.

2. Étape 2 : Spécifications Fonctionnelles (User Stories)

Le backlog du projet traduit les besoins métiers en fonctionnalités atomiques, tout en intégrant des **contraintes techniques de sécurité (OWASP)** et d'**accessibilité (RGAA)**.

Backlog des User Stories

- En tant qu' **Agent de terrain**, je veux **téléverser une fiche laboratoire afin d'extraire automatiquement les données physico-chimiques via OCR**.
- En tant qu' **Analyste Qualité**, je veux **visualiser les performances du modèle (Accuracy, F1-Score) afin de valider la fiabilité des prédictions en production**.
- En tant que **Responsable d'Exploitation**, je veux **authentifier chaque requête via une X-API-Key afin de garantir l'intégrité du système et l'isolation des données**.

Critères d'Acceptation (Agent de terrain)

Pour valider l'ingestion OCR, le système doit :

1. Permettre l'**extraction de 9 mesures physico-chimiques spécifiques**.
2. Réaliser un traitement asynchrone via le service externe OCR.space.
3. Parser les retours JSON via des expressions régulières pour alimenter le formulaire.

4. Respecter une Contrainte : **accessibilité numérique (RGAA)** incluant le **contraste visuel et la hiérarchie sémantique** pour les lecteurs d'écran.

3. Étape 3 : Décomposition en Composants Techniques

L'architecture est segmentée en modules indépendants pour garantir la scalabilité et la maintenance.

- **Le Cerveau (API FastAPI)** : Orchestre l'**exécution des routes /predict et /ocr**. Il assure la **validation native des types** via Pydantic, empêchant l'injection de données corrompues.
- **Le Stockage (PostgreSQL & Volumes)** : Gère la **séparation stricte** entre les **métadonnées relationnelles** (comptes, logs d'audit) et les **artefacts binaires lourds** (modèles .pkl stockés dans des volumes Docker).
- **L'Interface (Frontend Flask)** : Assure la **restitution visuelle des tableaux de bord**. Ce module gère l'**authentification experte** et **communiquant avec l'API** via des **requêtes HTTP sécurisées**.

4. Étape 4 : Architecture Technique et Flux de Données

Le système est entièrement conteneurisé. Pour répondre aux enjeux de sécurité réseau identifiés, nous implémentons une validation stricte du Host Header pour prévenir les attaques de **DNS Rebinding** entre FastAPI et MLflow.

Schéma Narratif de l'Architecture

Quatre conteneurs Docker interagissent :

1. **Container API (FastAPI)** : Point d'entrée unique.
2. **Container DB (PostgreSQL)** : Persistance des données et de l'Audit Trail.
3. **Container MLflow** : Tracking des modèles avec isolation réseau via Host Header Validation.
4. **Container ML-Training** : Environnement éphémère pour l'**entraînement des modèles XGBoost**.

Flux de l'Inférence

1. L'API vérifie la **validité de la X-API-Key** (hachée en SHA-256).
2. Une **action de filtrage SQL WHERE client_id = :id** est imposée pour garantir le **cloisonnement RGPD (Multi-Tenancy)**.
3. Le système applique les garde-fous sanitaires de l'OMS (pH, turbidité).
4. Le modèle est chargé en **Lazy Loading** pour optimiser la **mémoire vive**.

5. Étape 5 : Stack Technique et Justifications

Catégorie,Outil,Justification

Langage,Python,Standard industriel pour l'IA et le traitement asynchrone.

Framework,FastAPI,Performance et **sécurité (OpenAPI/Pydantic)**.

IA / MLOps,MLflow,Traçabilité et versionnement des artefacts.

Base de données,PostgreSQL,Robustesse et conformité pour l'Audit Trail.

DevOps,GitHub Actions,Automatisation des tests et de la **CI/CD**.

Focus Sécurité & Optimisation (Green IT)

1. **Lazy Loading** : Chargement différé du modèle pour économiser les ressources RAM au démarrage du conteneur.
2. **Hachage SHA-256** : Protection contre la faille OWASP "Broken Authentication" (clés API jamais stockées en clair).
3. **Garde-fous OMS & Green IT** : L'API rejette les échantillons non conformes (ex: pH < 6.5) avant l'appel au modèle. Cette approche évite de consommer des cycles CPU/GPU inutiles pour une inférence ML lourde, réduisant l'empreinte carbone.

6. Étape 6 : Stratégie de Réalisation et MLOps

Priorisation du Backlog (MVP First)

1. **Modèle de données** : Conception PostgreSQL pour stabiliser l'API.
2. **Intégration IA** : Encapsulation du modèle dans FastAPI avec MLflow.
3. **Dérisquage OCR** : Intégration prioritaire d'OCR.space pour gérer les latences tiers.

Cadre de Travail Agile

Méthodologie hybride **Scrum/Kanban** via Trello. Colonnes : *Backlog* , *To Do* , *In Progress* , *Code Review* , *Done* .

Pipeline CI/CD et Qualité

Le workflow GitHub Actions (ci.yml) exécute l' (**Action : Orange**) test PyTest à chaque commit :

- **Non-régression** : Validation du modèle sur water_std.csv. Le test échoue si le **F1-Score tombe sous 60%** .
- **Couverture de code** : L'objectif global est de 80%. Actuellement à 44%, nous priorisons une couverture entre **70% et 95% pour la logique métier critique** (api.py, ocr.py). Les scripts d'entraînement (experiment.py) restent à 0% par design en raison du découplage des conteneurs éphémères.

7. Observabilité et Maintien en Condition Opérationnelle (MCO)

Stack RED (Prometheus & Grafana)

Surveillance proactive des trois métriques vitales :

- **Rate** : Volume de trafic pour détecter les dénis de service.
- **Errors** : Taux de codes 5xx pour identifier les bugs applicatifs.
- **Duration** : Latence au 95ème centile pour garantir la réactivité.

Gestion des Incidents (Méthode DDCR)

1. **Détection** : Alerte automatique via Prometheus.
2. **Diagnostic** : Analyse des logs JSON structurés et de la table action_logs.
3. **Correction (Fallback OCR)** : En cas d'échec du service OCR.space (Timeout/Quota), l'API intercepte l'erreur et active un repli gracieux (message d'avertissement) pour éviter le crash du système.
4. **Retour d'expérience** : Documentation systématique dans le registre d'incidents.